

The WalkSAT algorithm

```
function WALKSAT(clauses, p, max-flips) returns a satisfying model or failure
  inputs: clauses, a set of clauses in propositional logic
         p, the probability of choosing to do a “random walk” move
         max-flips, number of flips allowed before giving up
  model ← a random assignment of true/false to the symbols in clauses
  for i = 1 to max-flips do
    if model satisfies clauses then return model
    clause ← a randomly selected clause from clauses that is false in model
    with probability p flip the value in model of a randomly selected symbol
      from clause
    else flip whichever symbol in clause maximizes the number of satisfied clauses
  return failure
```

Hard satisfiability problems

- Consider random 3-CNF sentences. e.g.,

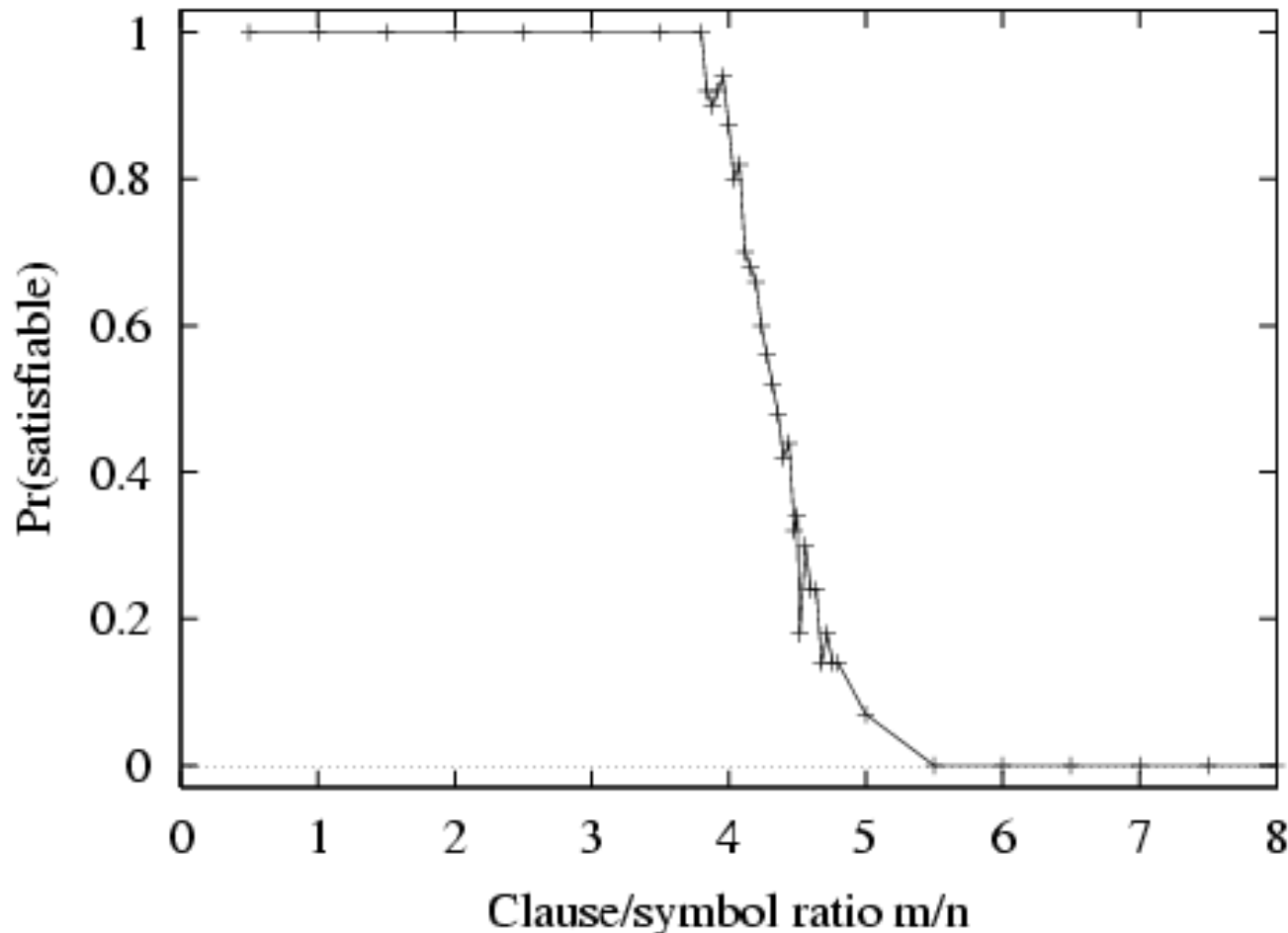
$$(\neg D \vee \neg B \vee C) \wedge (B \vee \neg A \vee \neg C) \wedge (\neg C \vee \neg B \vee E) \wedge (E \vee \neg D \vee B) \wedge (B \vee E \vee \neg C)$$

m = number of clauses

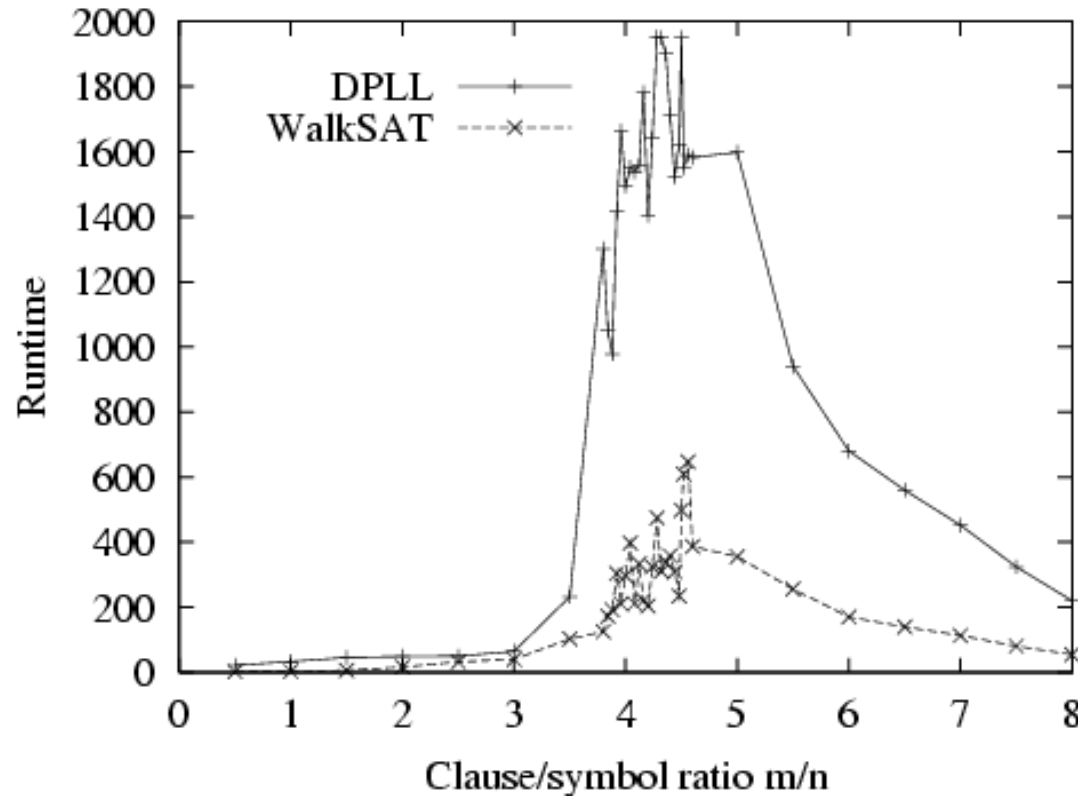
n = number of symbols

- Hard problems seem to cluster near $m/n = 4.3$ (critical point)

Hard satisfiability problems



Hard satisfiability problems



- Median runtime for 100 **satisfiable** random 3-CNF sentences, $n = 50$

Inference-based agents in the wumpus world

A wumpus-world agent using propositional logic:

$$\begin{aligned}& \neg P_{1,1} \\& \neg W_{1,1} \\& B_{x,y} \Leftrightarrow (P_{x,y+1} \vee P_{x,y-1} \vee P_{x+1,y} \vee P_{x-1,y}) \\& S_{x,y} \Leftrightarrow (W_{x,y+1} \vee W_{x,y-1} \vee W_{x+1,y} \vee W_{x-1,y}) \\& W_{1,1} \vee W_{1,2} \vee \dots \vee W_{4,4} \\& \neg W_{1,1} \vee \neg W_{1,2} \\& \neg W_{1,1} \vee \neg W_{1,3} \\& \dots\end{aligned}$$

$\Rightarrow$ 64 distinct proposition symbols, 155 sentences

function PL-WUMPUS-AGENT(*percept*) **returns** an *action*

inputs: *percept*, a list, [*stench*, *breeze*, *glitter*]

static: *KB*, initially containing the “physics” of the wumpus world
x, y, orientation, the agent’s position (init. [1,1]) and orient. (init. *right*)
visited, an array indicating which squares have been visited, initially *false*
action, the agent’s most recent action, initially null
plan, an action sequence, initially empty

update *x, y, orientation, visited* based on *action*

if *stench* **then** TELL(*KB*, $S_{x,y}$) **else** TELL(*KB*, $\neg S_{x,y}$)

if *breeze* **then** TELL(*KB*, $B_{x,y}$) **else** TELL(*KB*, $\neg B_{x,y}$)

if *glitter* **then** *action* $\leftarrow$ *grab*

else if *plan* is nonempty **then** *action* $\leftarrow$ POP(*plan*)

else if for some fringe square $[i,j]$, ASK(*KB*, $(\neg P_{i,j} \wedge \neg W_{i,j})$) is *true* or
for some fringe square $[i,j]$, ASK(*KB*, $(P_{i,j} \vee W_{i,j})$) is *false* **then do**
plan $\leftarrow$ A*-GRAPH-SEARCH(ROUTE-PB($[x,y]$, *orientation*, $[i,j]$, *visited*))
action $\leftarrow$ POP(*plan*)

else *action* $\leftarrow$ a randomly chosen move

return *action*

Summary

- Logical agents apply **inference** to a **knowledge base** to derive new information and make decisions
- Basic concepts of logic:
 - **syntax**: formal structure of **sentences**
 - **semantics**: **truth** of sentences wrt **models**
 - **entailment**: necessary truth of one sentence given another
 - **inference**: deriving sentences from other sentences
 - **soundness**: derivations produce only entailed sentences
 - **completeness**: derivations can produce all entailed sentences
- Wumpus world requires the ability to represent partial and negated information, reason by cases, etc.
- Resolution is complete for propositional logic
Forward, backward chaining are linear-time, complete for Horn clauses
- Propositional logic lacks expressive power



University of Colorado
Colorado Springs



University of Colorado

Boulder | Colorado Springs | Denver | Anschutz Medical Campus

Lecture 8: First Order Logic Part III

Russell and Norvig: Chapter 8

CS-4820/5820

Tu/Th 12:15 PM-1:30 PM

Room: Centennial Hall 106

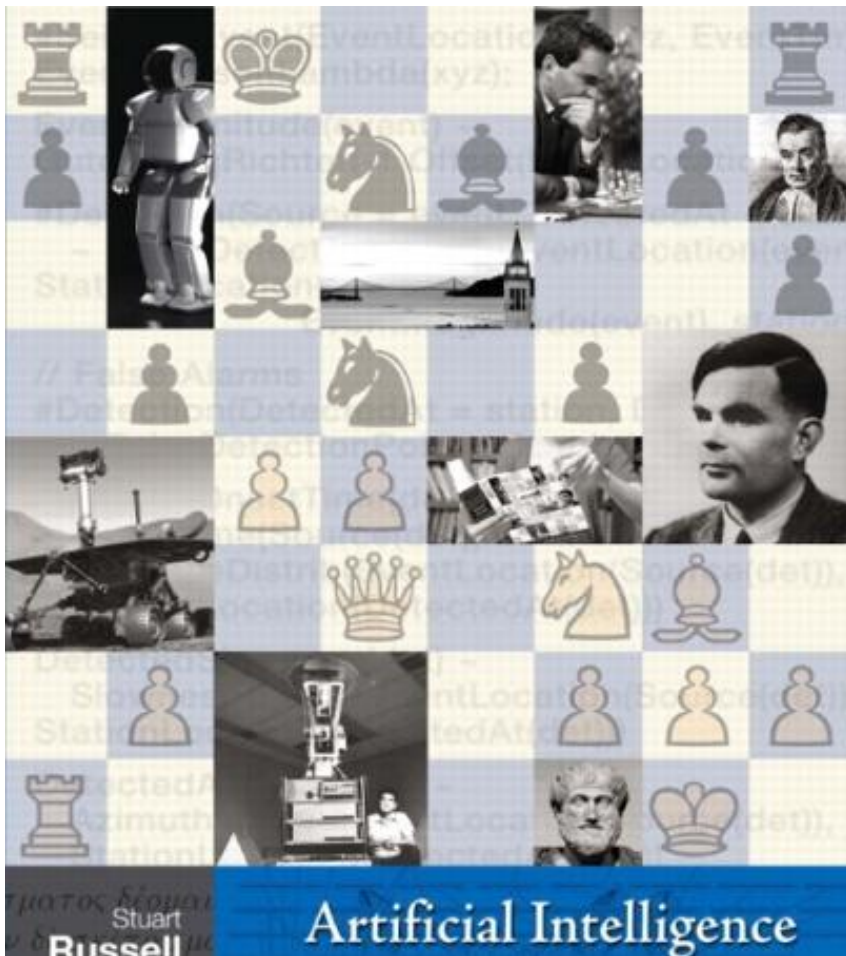
Instructor: Adham Atyabi

Office: Engineering 243

Office Hours: Mon 9:00 AM-14:00 PM.

Email: aatyabi@uccs.edu

Teaching Assistant: Ali Al Shami
(aalshami@uccs.edu)



Outline

- Why FOL?
- Syntax and semantics of FOL
- Using FOL
- Wumpus world in FOL
- Knowledge engineering in FOL

Pros and cons of propositional logic

- ☺ Propositional logic is **declarative**
- ☺ Propositional logic allows partial/disjunctive/negated information
 - (unlike most data structures and databases)
- ☺ Propositional logic is **compositional**:
 - meaning of $B_{1,1} \wedge P_{1,2}$ is derived from meaning of $B_{1,1}$ and of $P_{1,2}$
- ☺ Meaning in propositional logic is **context-independent**
 - (unlike natural language, where meaning depends on context)
- ☹ Propositional logic has very limited expressive power
 - (unlike natural language)
 - E.g., cannot say "pits cause breezes in adjacent squares"
 - ***except by writing one sentence for each square***

First-order logic

- propositional logic assumes the world contains **facts**,
- first-order logic (like natural language) assumes the world contains
 - **Objects**: people, houses, numbers, colors, baseball games, wars, ...
 - **Relations**: red, round, prime, brother of, bigger than, part of, comes between, ...
 - **Functions**: father of, best friend, one more than, plus, ...

Syntax of FOL: Basic elements

- Constants KingJohn, 2, NUS,...
- Predicates Brother, >,...
- Functions Sqrt, LeftLegOf,...
- Variables $x, y, a, b, \dots$
- Connectives $\neg, \Rightarrow, \wedge, \vee, \Leftrightarrow$
- Equality $=$
- Quantifiers $\forall, \exists$

Atomic sentences

Atomic sentence = *predicate* ($term_1, \dots, term_n$)
or $term_1 = term_2$

Term = *function* ($term_1, \dots, term_n$)
or *constant* or *variable*

- E.g., *Brother*(*KingJohn*, *RichardTheLionheart*)
Length(*LeftLegOf*(*Richard*)),
Length(*LeftLegOf*(*KingJohn*))

Complex sentences

- Complex sentences are made from atomic sentences using connectives

$$\neg S, S_1 \wedge S_2, S_1 \vee S_2, S_1 \Rightarrow S_2, S_1 \Leftrightarrow S_2,$$

E.g. *Sibling(KingJohn, Richard) $\Rightarrow$ Sibling(Richard, KingJohn)*

$$(1,2) \vee \leq (1,2)$$

$$(1,2) \wedge \neg > (1,2)$$

Or

Brother (R, J) $\wedge$ brother(J, R) $\vee$ $\neg$ King(Richard) $\Rightarrow$ king(john)

Truth in first-order logic

- Sentences are true with respect to a **model** and an **interpretation**
- Model contains objects (**domain elements**) and relations among them
- Interpretation specifies referents for
 - constant symbols** → **objects**
 - predicate symbols** → **relations**
 - function symbols** → **functional relations**
- An atomic sentence $predicate(term_1, \dots, term_n)$ is true iff the **objects** referred to by $term_1, \dots, term_n$ are in the **relation** referred to by $predicate$

Interpretation

An interpretation I is defined by a **mapping** to the **domain of discourse D or relations on D**

- **domain of discourse:** a set of objects in the world we represent and refer to;

An interpretation I maps:

- Constant symbols to objects in D

$$I(\text{John}) = \text{stick figure}$$

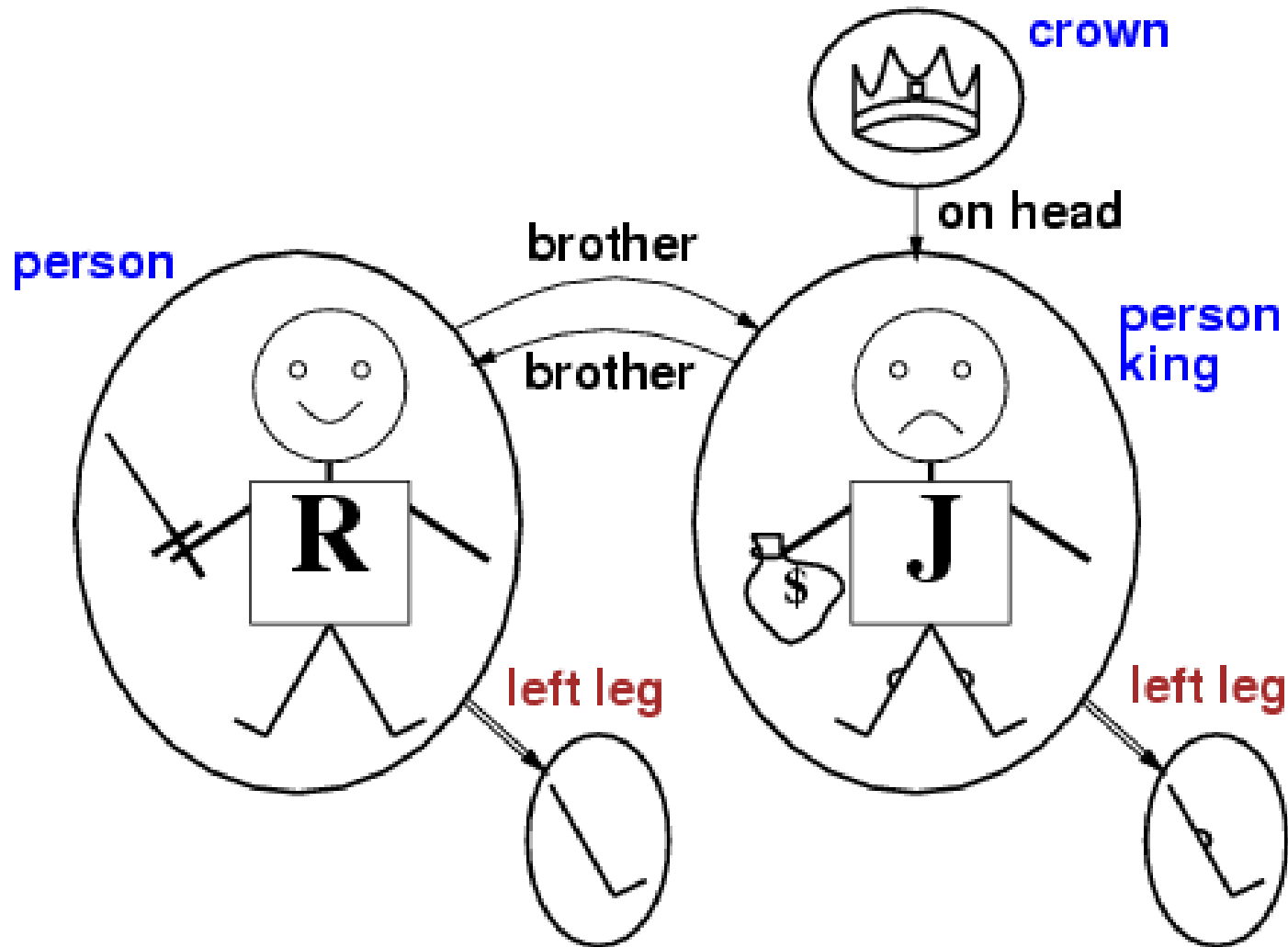
- Predicate symbols to relations, properties on D

$$I(\text{brother}) = \left\{ \langle \text{stick figure}, \text{robot} \rangle; \langle \text{stick figure}, \text{stick figure} \rangle; \dots \right\}$$

- Function symbols to functional relations on D

$$I(\text{father-of}) = \left\{ \langle \text{stick figure} \rangle \rightarrow \text{stick figure}; \quad \langle \text{stick figure} \rangle \rightarrow \text{robot}; \dots \right\}$$

Models for FOL: Example



Universal quantification

- $\forall \langle \text{variables} \rangle \langle \text{sentence} \rangle$

Everyone at UCCS is smart: $\forall x \text{ At}(x, \text{UCCS}) \Rightarrow \text{Smart}(x)$

- $\forall x P$ is true in a model m if P is true with x being each possible object in the model
- Roughly speaking, equivalent to the **conjunction** of **instantiations** of P
 - $\text{At}(\text{KingJohn}, \text{UCCS}) \Rightarrow \text{Smart}(\text{KingJohn})$
 - $\wedge \text{At}(\text{Richard}, \text{UCCS}) \Rightarrow \text{Smart}(\text{Richard})$
 - $\wedge \text{At}(\text{UCCS}, \text{UCCS}) \Rightarrow \text{Smart}(\text{UCCS})$
 - $\wedge \dots$

A common mistake to avoid

- Typically, $\Rightarrow$ is the main connective with $\forall$

- Common mistake: using $\wedge$ as the main connective with $\forall$:

$$\forall x \text{ At}(x, \text{UCCS}) \wedge \text{Smart}(x)$$

means “Everyone is at UCCS and everyone is smart”

Existential quantification

- $\exists \langle \text{variables} \rangle \langle \text{sentence} \rangle$
- Someone at UCCS is smart:
 $\exists x \text{ At}(x, \text{UCCS}) \wedge \text{Smart}(x)$
- $\exists x P$ is true in a model m iff P is true with x being some possible object in the model
- Roughly speaking, equivalent to the **disjunction** of **instantiations** of P
 - $\text{At}(\text{KingJohn}, \text{UCCS}) \wedge \text{Smart}(\text{KingJohn})$
 - $\vee \text{ At}(\text{Richard}, \text{UCCS}) \wedge \text{Smart}(\text{Richard})$
 - $\vee \text{ At}(\text{UCCS}, \text{UCCS}) \wedge \text{Smart}(\text{UCCS})$
 - $\vee \dots$

Another common mistake to avoid

- Typically, $\wedge$ is the main connective with $\exists$
- Common mistake: using $\Rightarrow$ as the main connective with $\exists$:

$$\exists x \text{ At}(x, \text{UCCS}) \Rightarrow \text{Smart}(x)$$

is true if there is anyone who is not at UCCS!

Properties of quantifiers

- $\forall x \forall y$ is the same as $\forall y \forall x$
- $\exists x \exists y$ is the same as $\exists y \exists x$
- $\exists x \forall y$ is **not** the same as $\forall y \exists x$
- $\exists x \forall y \text{ Loves}(x,y)$
 - “There is a person who loves everyone in the world”
- $\forall y \exists x \text{ Loves}(x,y)$
 - “Everyone in the world is loved by at least one person”
- **Quantifier duality**: each can be expressed using the other
- $\forall x \text{ Likes}(x, \text{IceCream}) \quad \neg \exists x \neg \text{Likes}(x, \text{IceCream})$
- $\exists x \text{ Likes}(x, \text{Broccoli}) \quad \neg \forall x \neg \text{Likes}(x, \text{Broccoli})$

Equality

- $term_1 = term_2$ is true under a given interpretation if and only if $term_1$ and $term_2$ refer to the same object
- E.g., definition of *Sibling* in terms of *Parent*:
$$\forall x, y \text{ Sibling}(x, y) \Leftrightarrow [\neg(x = y) \wedge \exists m, f \neg (m = f) \wedge \text{Parent}(m, x) \wedge \text{Parent}(f, x) \wedge \text{Parent}(m, y) \wedge \text{Parent}(f, y)]$$

Using FOL

The kinship domain:

- Brothers are siblings

$$\forall x,y \text{ Brother}(x,y) \Leftrightarrow \text{Sibling}(x,y)$$

- One's mother is one's female parent

$$\forall m,c \text{ Mother}(c) = m \Leftrightarrow (\text{Female}(m) \wedge \text{Parent}(m,c))$$

- “Sibling” is symmetric

$$\forall x,y \text{ Sibling}(x,y) \Leftrightarrow \text{Sibling}(y,x)$$

Using FOL

The set domain:

1. $\forall s \text{ Set}(s) \Leftrightarrow (s = \{\}) \vee (\exists x, s_2 \text{ Set}(s_2) \wedge s = \{x|s_2\})$
2. $\neg \exists x, s \{x|s\} = \{\}$
3. $\forall x, s x \in s \Leftrightarrow s = \{x|s\}$
4. $\forall x, s x \in s \Leftrightarrow [\exists y, s_2 \{ (s = \{y|s_2\} \wedge (x = y \vee x \in s_2))]$
5. $\forall s_1, s_2 s_1 \subseteq s_2 \Leftrightarrow (\forall x x \in s_1 \Rightarrow x \in s_2)$
6. $\forall s_1, s_2 (s_1 = s_2) \Leftrightarrow (s_1 \subseteq s_2 \wedge s_2 \subseteq s_1)$
7. $\forall x, s_1, s_2 x \in (s_1 \cap s_2) \Leftrightarrow (x \in s_1 \wedge x \in s_2)$
8. $\forall x, s_1, s_2 x \in (s_1 \cup s_2) \Leftrightarrow (x \in s_1 \vee x \in s_2)$

Interacting with FOL KBs

- Suppose a wumpus-world agent is using an FOL KB and perceives a smell and a breeze (but no glitter) at $t=5$:

Tell(KB, Percept([Smell, Breeze, None], 5))

Ask(KB, $\exists a$ BestAction($a, 5$)) returns true or false

i.e., does the KB entail some best action at $t=5$?

ASKVARS($\exists a$ BestAction($a, 5$)) returns a respond, like {a/left, a/down, a/Shoot}

Answer: *Yes*, {a/Shoot} $\leftarrow$ **substitution** (binding list)

- Given a sentence S and a substitution σ ,
- $S\sigma$ denotes the result of plugging σ into S ; e.g.,
 $S = \text{Smarter}(x, y)$
 $\sigma = \{x/\text{Hillary}, y/\text{Bill}\}$
 $S\sigma = \text{Smarter}(\text{Hillary}, \text{Bill})$
- Ask**(KB, S) returns some/all σ such that $\text{KB} \models \sigma$

Knowledge base for the wumpus world

- Perception

- $\forall t, s, b \text{ Percept}([s, b, \text{Glitter}], t) \Rightarrow \text{Glitter}(t)$

- Reflex

- $\forall t \text{ Glitter}(t) \Rightarrow \text{BestAction}(\text{Grab}, t)$

Deducing hidden properties

- $\forall x,y,a,b \text{ Adjacent}([x,y],[a,b]) \Leftrightarrow$
 $[a,b] \in \{[x+1,y], [x-1,y],[x,y+1],[x,y-1]\}$

Properties of squares:

- $\forall s,t \text{ At}(\text{Agent},s,t) \wedge \text{Breeze}(t) \Rightarrow \text{Breezy}(s)$

Squares are breezy near a pit:

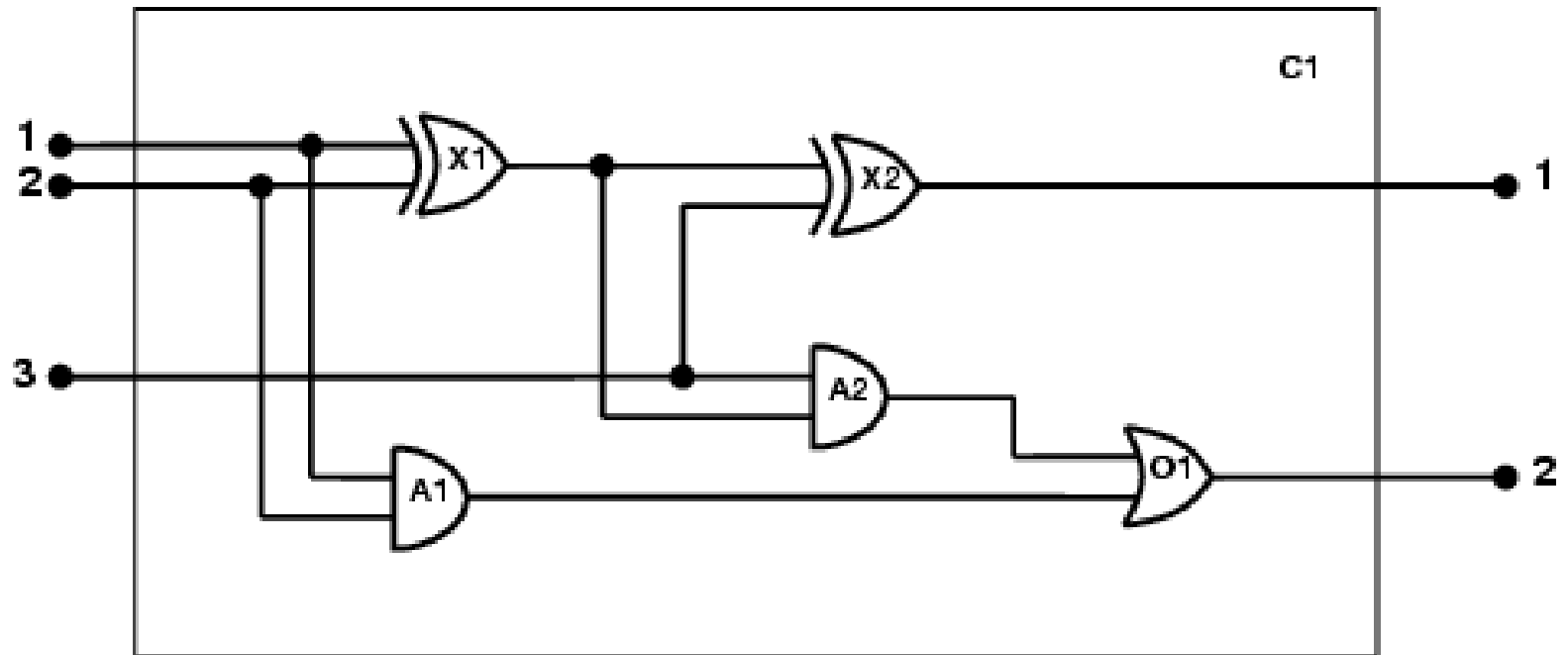
- **Diagnostic** rule---infer cause from effect
 $\forall s \text{ Breezy}(s) \Rightarrow \exists r \text{ Adjacent}(r,s) \wedge \text{Pit}(r)$
- **Causal** rule---infer effect from cause
 $\forall r \text{ Pit}(r) \Rightarrow [\forall s \text{ Adjacent}(r,s) \Rightarrow \text{Breezy}(s)]$

Knowledge engineering in FOL

1. Identify the task
2. Assemble the relevant knowledge
3. Decide on a vocabulary of predicates, functions, and constants
4. Encode general knowledge about the domain
5. Encode a description of the specific problem instance
6. Pose queries to the inference procedure and get answers
7. Debug the knowledge base

The electronic circuits domain

One-bit full adder



The electronic circuits domain

1. Identify the task

- Does the circuit actually add properly? (circuit verification)

2. Assemble the relevant knowledge

- Composed of wires and gates; Types of gates (AND, OR, XOR, NOT)
- Irrelevant: size, shape, color, cost of gates

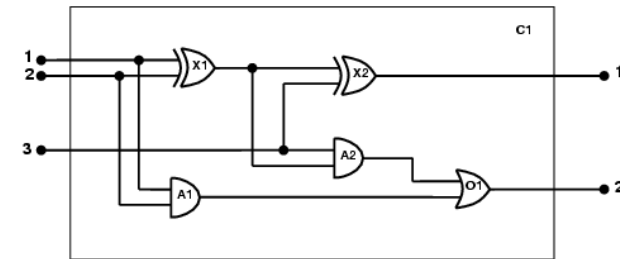
3. Decide on a vocabulary

- Alternatives:
 $Type(X_1) = XOR$
 $Type(X_1, XOR)$
 $XOR(X_1)$

The electronic circuits domain

4. Encode general knowledge of the domain

- $\forall t_1, t_2 \text{ Connected}(t_1, t_2) \Rightarrow \text{Signal}(t_1) = \text{Signal}(t_2)$
- $\forall t \text{ Signal}(t) = 1 \vee \text{Signal}(t) = 0$
- $1 \neq 0$
- $\forall t_1, t_2 \text{ Connected}(t_1, t_2) \Rightarrow \text{Connected}(t_2, t_1)$
- $\forall g \text{ Type}(g) = \text{OR} \Rightarrow \text{Signal}(\text{Out}(1, g)) = 1 \Leftrightarrow \exists n \text{ Signal}(\text{In}(n, g)) = 1$
- $\forall g \text{ Type}(g) = \text{AND} \Rightarrow \text{Signal}(\text{Out}(1, g)) = 0 \Leftrightarrow \exists n \text{ Signal}(\text{In}(n, g)) = 0$
- $\forall g \text{ Type}(g) = \text{XOR} \Rightarrow \text{Signal}(\text{Out}(1, g)) = 1 \Leftrightarrow \text{Signal}(\text{In}(1, g)) \neq \text{Signal}(\text{In}(2, g))$
- $\forall g \text{ Type}(g) = \text{NOT} \Rightarrow \text{Signal}(\text{Out}(1, g)) \neq \text{Signal}(\text{In}(1, g))$

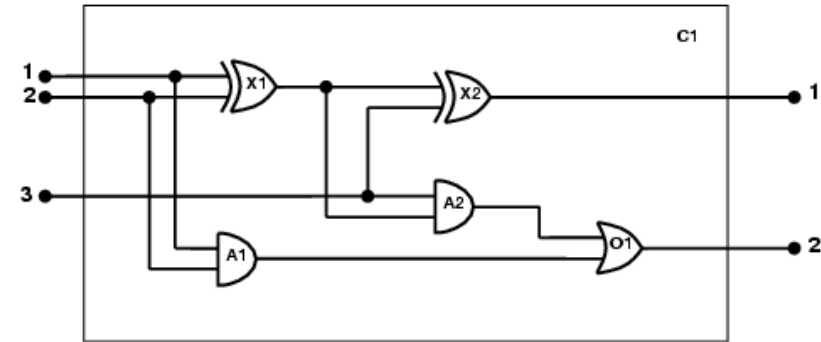


The electronic circuits domain

5. Encode the specific problem instance

Type(X_1) = XOR Type(X_2) = XOR
Type(A_1) = AND Type(A_2) = AND
Type(O_1) = OR

Connected(Out(1, X_1),In(1, X_2))
Connected(Out(1, X_1),In(2, A_2))
Connected(Out(1, A_2),In(1, O_1))
Connected(Out(1, A_1),In(2, O_1))
Connected(Out(1, X_2),Out(1, C_1))
Connected(Out(1, O_1),Out(2, C_1))



Connected(In(1, C_1),In(1, X_1))
Connected(In(1, C_1),In(1, A_1))
Connected(In(2, C_1),In(2, X_1))
Connected(In(2, C_1),In(2, A_1))
Connected(In(3, C_1),In(2, X_2))
Connected(In(3, C_1),In(1, A_2))

The electronic circuits domain

6. Pose queries to the inference procedure

What are the possible sets of values of all the terminals for the adder circuit?

$$\begin{aligned} \exists i_1, i_2, i_3, o_1, o_2 \text{ Signal(In}(1, C_1)) = i_1 \wedge \text{Signal(In}(2, C_1)) = i_2 \wedge \\ \text{Signal(In}(3, C_1)) = i_3 \wedge \text{Signal(Out}(1, C_1)) = o_1 \wedge \\ \text{Signal(Out}(2, C_1)) = o_2 \end{aligned}$$

7. Debug the knowledge base

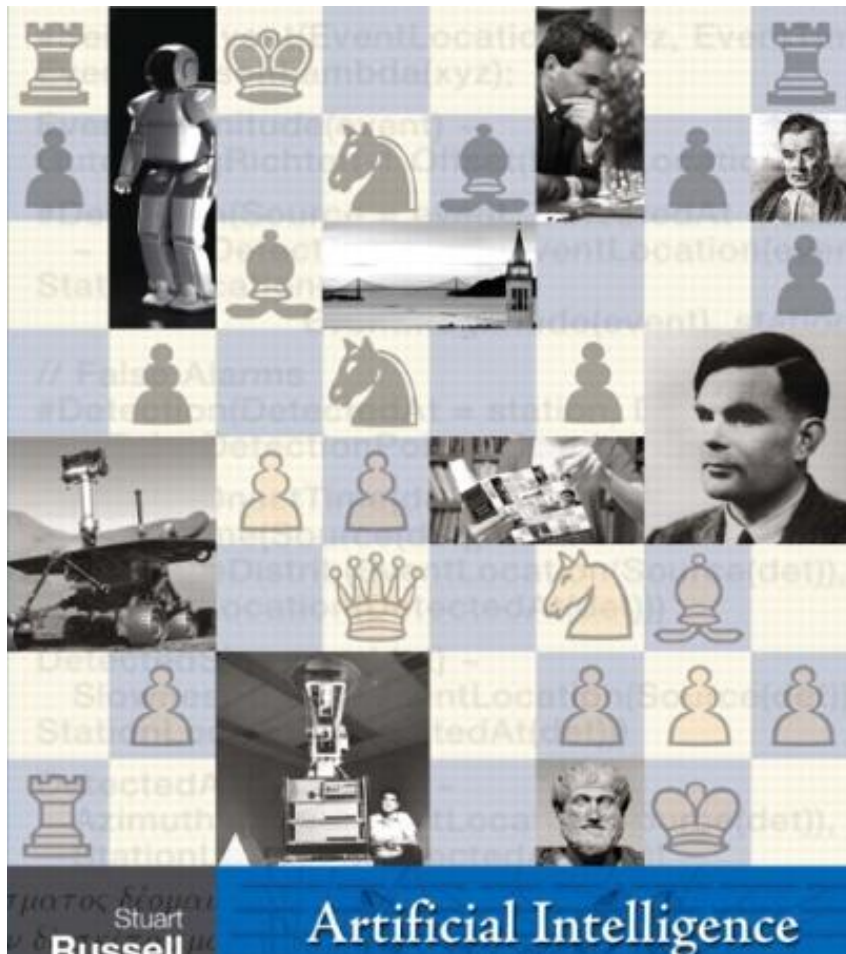
May have omitted assertions like $1 \neq 0$

Summary

- First-order logic:
 - objects and relations are semantic primitives
 - syntax: constants, functions, predicates, equality, quantifiers
- Increased expressive power: sufficient to define wumpus world

Lecture 8: Inference in first-order logic Part IV

Russell and Norvig: Chapter 9



CS-4820/5820

Tu/Th 12:15 PM-1:30 PM

Room: Centennial Hall 106

Instructor: Adham Atyabi

Office: Engineering 243

Office Hours: Mon 9:00 AM-14:00 PM.

Email: aatyabi@uccs.edu

**Teaching Assistant: Ali Al Shami
(aalshami@uccs.edu)**

Outline

- Reducing first-order inference to propositional inference
- Unification
- Generalized Modus Ponens
- Forward chaining
- Backward chaining
- Resolution

Creating a knowledge Base

- ▶ Identify the Task
- ▶ Assemble the relevant knowledge
- ▶ Decide on a vocabulary of predicates, functions and constants
- ▶ Encode general knowledge about the domain (rules)
- ▶ Encode a description of the problem
- ▶ Pose queries to the inference procedure and get answers
- ▶ Debug the KB

Universal instantiation (UI)

- Every instantiation of a universally quantified sentence is entailed by it:

$$\frac{\forall v \ \alpha}{\text{Subst}(\{v/g\}, \alpha)}$$

for any variable v and ground term g

- E.g., lets say we have this rule,
 $\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$ so we can infer followings:

$\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John})$

$\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \Rightarrow \text{Evil}(\text{Richard})$

$\text{King}(\text{Father}(\text{John})) \wedge \text{Greedy}(\text{Father}(\text{John})) \Rightarrow \text{Evil}(\text{Father}(\text{John}))$

.

.

- Remember that from previous lectures, we do have algorithms to solve $\text{King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$. The thing we don't have a way to solve right now is how to deal with is $\forall x$ term.

Existential instantiation (EI)

- For any sentence α , variable v , and constant symbol k that does **not** appear elsewhere in the knowledge base:

$$\frac{\exists v \alpha}{\text{Subst}(\{v/k\}, \alpha)}$$

- E.g., $\exists x \text{Crown}(x) \wedge \text{OnHead}(x, \text{John})$ yields:

$$\text{Crown}(C_1) \wedge \text{OnHead}(C_1, \text{John})$$

provided C_1 is a new constant symbol, called a **Skolem constant**

- Skolem constant** is a new variables that represent new inference

Reduction to propositional inference

Suppose the KB contains just the following:

$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$

$\text{King}(\text{John})$

$\text{Greedy}(\text{John})$

$\text{Brother}(\text{Richard}, \text{John})$

- Instantiating the universal sentence in **all possible** ways, we have:

$\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John})$

$\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \Rightarrow \text{Evil}(\text{Richard})$

$\text{King}(\text{John})$

$\text{Greedy}(\text{John})$

$\text{Brother}(\text{Richard}, \text{John})$

- The new KB is **propositionalized**: proposition symbols are $\text{King}(\text{John})$, $\text{Greedy}(\text{John})$, $\text{Evil}(\text{John})$, $\text{King}(\text{Richard})$, etc.

Reduction contd.

- Every FOL KB can be propositionalized so as to preserve entailment
- A ground sentence is entailed by new KB if entailed by original KB
- Idea: propositionalize KB and query, apply resolution, return result
- Problem: with function symbols, there are infinitely many ground terms,
 - e.g., *Father(Father(Father(John)))*

Reduction contd.

Theorem: Herbrand (1930). If a sentence α is entailed by an FOL KB, it is entailed by a **finite** subset of the propositionalized KB

Idea: For $n = 0$ to ∞ do
 create a propositional KB by instantiating with depth- n terms
 see if α is entailed by this KB

Problem: works if α is entailed, loops if α is not entailed

Theorem: Turing (1936), Church (1936) Entailment for FOL is **semidecidable** (algorithms exist that say yes to every entailed sentence, but no algorithm exists that also says no to every nonentailed sentence.)

Problems with propositionalization

- Propositionalization seems to generate lots of irrelevant sentences.
- E.g., from:
 $\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$
 $\text{King}(\text{John})$
 $\forall y \text{ Greedy}(y)$
 $\text{Brother}(\text{Richard}, \text{John})$
- it seems obvious that *Evil(John)*, but propositionalization produces lots of facts such as *Greedy(Richard)* that are irrelevant
- With p k -array predicates and n constants, there are $p \cdot n^k$ instantiations.

Unification

- We can get the inference immediately if we can find a substitution θ such that $King(x)$ and $Greedy(x)$ match $King(John)$ and $Greedy(y)$

$\theta = \{x/John, y/John\}$ works

- $Unify(\alpha, \beta) = \theta$ if $\alpha\theta = \beta\theta$

p	q	θ
Knows(John,x)	Knows(John,Jane)	
Knows(John,x)	Knows(y,OJ)	
Knows(John,x)	Knows(y,Mother(y))	
Knows(John,x)	Knows(x,OJ)	

- Standardizing apart** eliminates overlap of variables, e.g., Knows(z_{17} , OJ)

Unification

- We can get the inference immediately if we can find a substitution θ such that $King(x)$ and $Greedy(x)$ match $King(John)$ and $Greedy(y)$

$\theta = \{x/John, y/John\}$ works

- $Unify(\alpha, \beta) = \theta$ if $\alpha\theta = \beta\theta$

p	q	θ
Knows(John,x)	Knows(John,Jane)	$\{x/Jane\}$
Knows(John,x)	Knows(y,OJ)	
Knows(John,x)	Knows(y,Mother(y))	
Knows(John,x)	Knows(x,OJ)	

- Standardizing apart** eliminates overlap of variables, e.g., $Knows(z_{17}, OJ)$

Unification

- We can get the inference immediately if we can find a substitution θ such that $King(x)$ and $Greedy(x)$ match $King(John)$ and $Greedy(y)$

$\theta = \{x/John, y/John\}$ works

- $Unify(\alpha, \beta) = \theta$ if $\alpha\theta = \beta\theta$

p	q	θ
Knows(John,x)	Knows(John,Jane)	$\{x/Jane\}$
Knows(John,x)	Knows(y,OJ)	$\{x/OJ, y/John\}$
Knows(John,x)	Knows(y,Mother(y))	
Knows(John,x)	Knows(x,OJ)	

- Standardizing apart** eliminates overlap of variables, e.g., $Knows(z_{17}, OJ)$

Unification

- We can get the inference immediately if we can find a substitution θ such that $King(x)$ and $Greedy(x)$ match $King(John)$ and $Greedy(y)$

$\theta = \{x/John, y/John\}$ works

- $Unify(\alpha, \beta) = \theta$ if $\alpha\theta = \beta\theta$

p	q	θ
Knows(John,x)	Knows(John,Jane)	$\{x/Jane\}$
Knows(John,x)	Knows(y,OJ)	$\{x/OJ, y/John\}$
Knows(John,x)	Knows(y,Mother(y))	$\{y/John, x/Mother(John)\}$
Knows(John,x)	Knows(x,OJ)	

- Standardizing apart** eliminates overlap of variables, e.g., $Knows(z_{17}, OJ)$

Unification

- We can get the inference immediately if we can find a substitution θ such that $King(x)$ and $Greedy(x)$ match $King(John)$ and $Greedy(y)$

$\theta = \{x/John, y/John\}$ works

- $Unify(\alpha, \beta) = \theta$ if $\alpha\theta = \beta\theta$

p	q	θ
Knows(John,x)	Knows(John,Jane)	$\{x/Jane\}$
Knows(John,x)	Knows(y,OJ)	$\{x/OJ, y/John\}$
Knows(John,x)	Knows(y,Mother(y))	$\{y/John, x/Mother(John)\}$
Knows(John,x)	Knows(x,OJ)	$\{fail\}$

- Standardizing apart** eliminates overlap of variables, e.g., $Knows(z_{17}, OJ)$

Unification

- To unify $Knows(John, x)$ and $Knows(y, z)$,
 $\theta = \{y/John, x/z\}$ or $\theta = \{y/John, x/John, z/John\}$
- The first unifier is **more general** than the second.
- There is a single **most general unifier** (MGU) that is unique up to renaming of variables.
 $MGU = \{y/John, x/z\}$

The unification algorithm

function UNIFY(x, y, θ) **returns** a substitution to make x and y identical

inputs: x , a variable, constant, list, or compound

y , a variable, constant, list, or compound

θ , the substitution built up so far

if $\theta = \text{failure}$ **then return failure**

else if $x = y$ **then return** θ

else if VARIABLE?(x) **then return** UNIFY-VAR(x, y, θ)

else if VARIABLE?(y) **then return** UNIFY-VAR(y, x, θ)

else if COMPOUND?(x) **and** COMPOUND?(y) **then**

return UNIFY(ARGS[x], ARGS[y], UNIFY(OP[x], OP[y], θ))

else if LIST?(x) **and** LIST?(y) **then**

return UNIFY(REST[x], REST[y], UNIFY(FIRST[x], FIRST[y], θ))

else return failure

The unification algorithm

function UNIFY-VAR(var, x, θ) **returns** a substitution

inputs: var , a variable

x , any expression

θ , the substitution built up so far

if $\{var/val\} \in \theta$ **then return** UNIFY(val, x, θ)

else if $\{x/val\} \in \theta$ **then return** UNIFY(var, val, θ)

else if OCCUR-CHECK?(var, x) **then return** failure

else return add $\{var/x\}$ to θ

Generalized Modus Ponens (GMP)

$$\frac{p_1', p_2', \dots, p_n', (p_1 \wedge p_2 \wedge \dots \wedge p_n \Rightarrow q)}{q\theta} \quad \text{where } p_i'\theta = p_i \theta \text{ for all } i$$

p_1' is <i>King(John)</i>	p_1 is <i>King(x)</i>
p_2' is <i>Greedy(y)</i>	p_2 is <i>Greedy(x)</i>
θ is {x/John,y/John}	q is <i>Evil(x)</i>
$q \theta$ is <i>Evil(John)</i>	

- GMP used with KB of **definite clauses** (**exactly** one positive literal)
- All variables assumed universally quantified

Inference unification

$UNIFY(p, q) = \theta$ Where $SUBST(\theta, p) = SUBST(\theta, q)$

For example:

- ▶ We ask $ASKVARS(Knows(John, x))$ (Whom does John know?)
- ▶ $UNIFY(Knows(John, x), Knows(John, Jane)) = \{x/Jane\}$
- ▶ $UNIFY(Knows(John, x), Knows(y, Bill)) = \{y/John, x/Bill\}$
- ▶ $UNIFY(Knows(John, x), Knows(y, Mother(y))) = \{y/John, x/Mother(John)\}$

Algorithm in the book (goes variable by variable recursively unifying)

Soundness of GMP

- Need to show that

$$p_1', \dots, p_n', (p_1 \wedge \dots \wedge p_n \Rightarrow q) \models q\theta$$

provided that $p_i'\theta = p_i\theta$ for all i

- Lemma: For any sentence p , we have $p \models p\theta$ by UI

1. $(p_1 \wedge \dots \wedge p_n \Rightarrow q) \models (p_1 \wedge \dots \wedge p_n \Rightarrow q)\theta = (p_1\theta \wedge \dots \wedge p_n\theta \Rightarrow q\theta)$
2. $p_1', \dots, p_n' \models p_1' \wedge \dots \wedge p_n' \models p_1'\theta \wedge \dots \wedge p_n'\theta$
3. From 1 and 2, $q\theta$ follows by ordinary Modus Ponens

Example knowledge base

- The law says that it is a crime for an American to sell weapons to hostile nations. The country Nono, an enemy of America, has some missiles, and all of its missiles were sold to it by Colonel West, who is American.
- Prove that Col. West is a criminal

Example knowledge base contd.

... it is a crime for an American to sell weapons to hostile nations:

R1: $American(x) \wedge Weapon(y) \wedge Sells(x,y,z) \wedge Hostile(z) \Rightarrow Criminal(x)$

Nono ... has some missiles, i.e., $\exists x Owns(Nono,x) \wedge Missile(x)$:

R2: $Owns(Nono,M_1) \wedge Missile(M_1)$

... all of its missiles were sold to it by Colonel West

R3: $Missile(x) \wedge Owns(Nono,x) \Rightarrow Sells(West,x,Nono)$

Missiles are weapons:

R4: $Missile(x) \Rightarrow Weapon(x)$

An enemy of America counts as "hostile":

R5: $Enemy(x,America) \Rightarrow Hostile(x)$

West, who is American ...

R6: $American(West)$

The country Nono, an enemy of America ...

R7: $Enemy(Nono,America)$

Example knowledge base contd.

R1: American(x) $\wedge$ Weapon(y) $\wedge$ Sells(x,y,z) $\wedge$ Hostile(z) $\Rightarrow$ Criminal(x)

R2: Owns(Nono,M₁) and Missile(M₁)

R3: Missile(x) $\wedge$ Owns(Nono,x) $\Rightarrow$ Sells(West,x,Nono)

R4: Missile(x) $\Rightarrow$ Weapon(x)

R5: Enemy(x,America) $\Rightarrow$ Hostile(x)

R6: American(West)

R7: Enemy(Nono,America)

Iteration 1

- R3 satisfied with {x/M1} and R8 will be generated and added to KB
 - R8: Sells(West,M1,Nono)
- R4 satisfied with {x/M1} and R9 will be generated and added to KB
 - R9: Weapon(M1)
- R5 satisfied with {x/Nono} and R10 is generated and added to KB
 - R10: Hostile(Nono)

Iteration 2

- Using R8 to R10, we can infer that R1 is satisfied with {x/west,y/M1,z/Nono} and Criminal(West) is added to KB.

Inference in first order Logic

- ▶ Once we have facts that evaluate to T or F
- ▶ We can apply Forward Chaining, Backwards Chaining and Resolution
- ▶ The key is to understand Unification
- ▶ Very similar to Logical agents.

Forward chaining algorithm

function FOL-FC-ASK(KB, α) **returns** a substitution or *false*

repeat until *new* is empty

$new \leftarrow \{\}$

for each sentence r **in** KB **do**

$(p_1 \wedge \dots \wedge p_n \Rightarrow q) \leftarrow \text{STANDARDIZE-APART}(r)$

for each θ **such that** $(p_1 \wedge \dots \wedge p_n)\theta = (p'_1 \wedge \dots \wedge p'_n)\theta$

for some $p'_1, \dots, p'_n$ **in** KB

$q' \leftarrow \text{SUBST}(\theta, q)$

if q' is not a renaming of a sentence already in KB or *new* **then do**

add q' **to** *new*

$\phi \leftarrow \text{UNIFY}(q', \alpha)$

if ϕ is not *fail* **then return** ϕ

add *new* **to** KB

return *false*

Forward chaining proof

R2: Owns(Nono, M₁) and Missile(M₁)

R6: American(West)

R7: Enemy(Nono, America)

American(West)

Missile(M1)

Owns(Nono, M1)

Enemy(Nono, America)

Forward chaining proof

R2: Owns(Nono, M₁) and Missile(M₁)

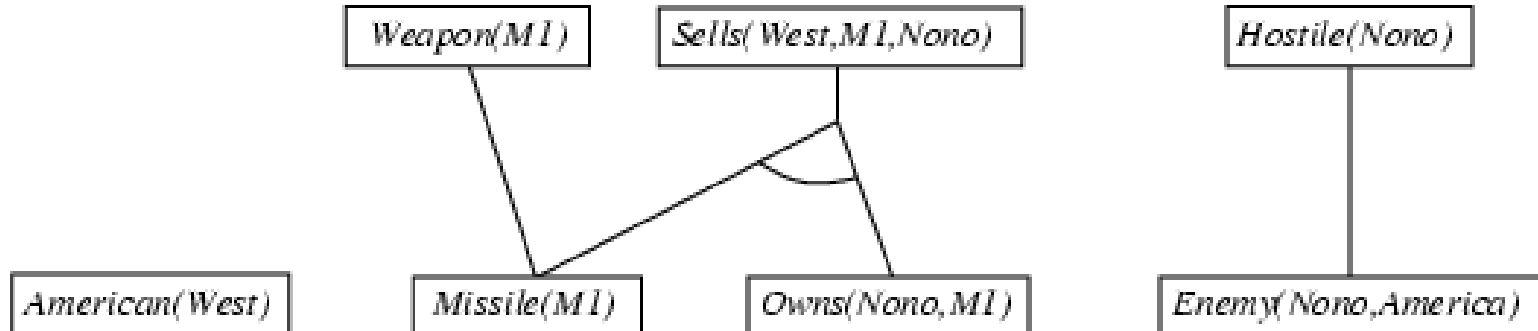
R6: American(West)

R7: Enemy(Nono, America)

R3: Missile(x) $\wedge$ Owns(Nono, x) $\Rightarrow$ Sells(West, x, Nono)

R4: Missile(x) $\Rightarrow$ Weapon(x)

R5: Enemy(x, America) $\Rightarrow$ Hostile(x)



Forward chaining proof

R2: $\text{Owns}(\text{Nono}, M_1)$ and $\text{Missile}(M_1)$

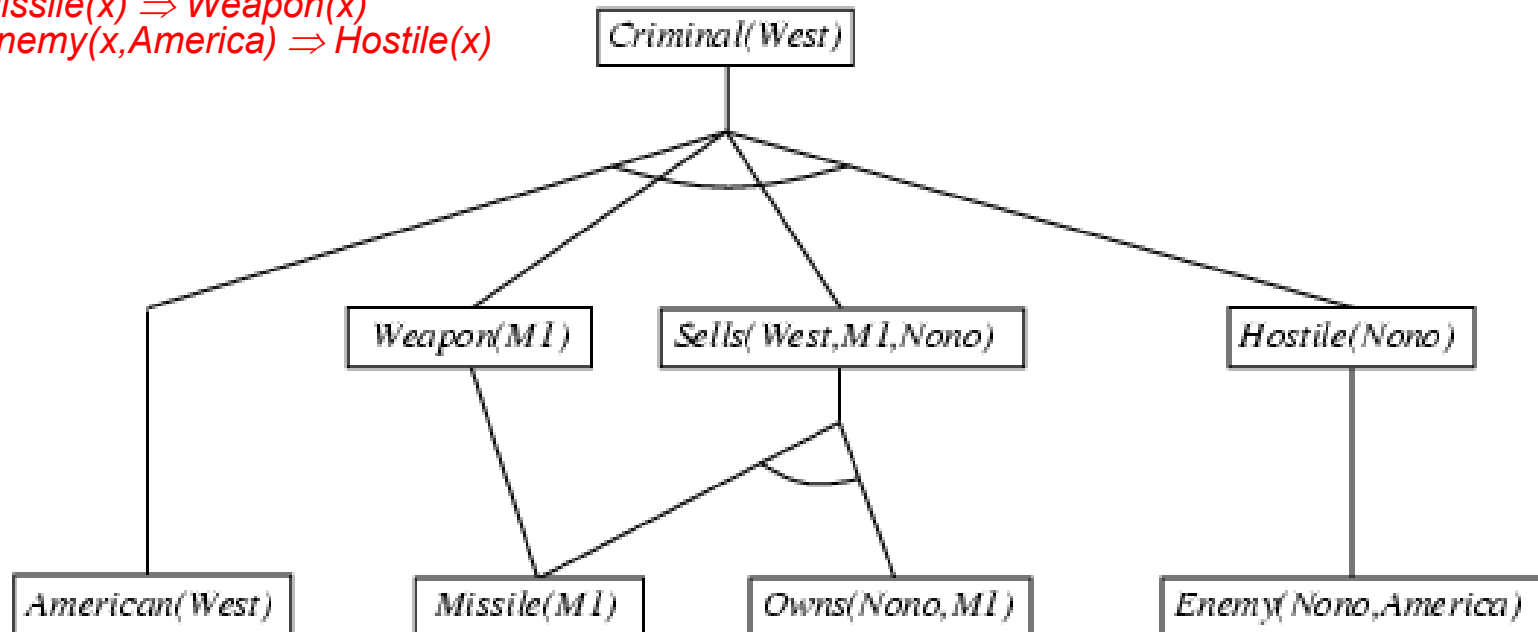
R6: $\text{American}(\text{West})$

R7: $\text{Enemy}(\text{Nono}, \text{America})$

R3: $\text{Missile}(x) \wedge \text{Owns}(\text{Nono}, x) \Rightarrow \text{Sells}(\text{West}, x, \text{Nono})$

R4: $\text{Missile}(x) \Rightarrow \text{Weapon}(x)$

R5: $\text{Enemy}(x, \text{America}) \Rightarrow \text{Hostile}(x)$



R1: $\text{American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}(x, y, z) \wedge \text{Hostile}(z) \Rightarrow \text{Criminal}(x)$

Properties of forward chaining

- Sound and complete for first-order definite clauses
- **Datalog** = first-order definite clauses + **no functions**
- FC terminates for Datalog in finite number of iterations
- May not terminate in general if α is not entailed
- This is unavoidable: entailment with definite clauses is semidecidable

Efficiency of forward chaining

Incremental forward chaining: no need to match a rule on iteration k if a premise wasn't added on iteration $k-1$

⇒ match each rule whose premise contains a newly added positive literal

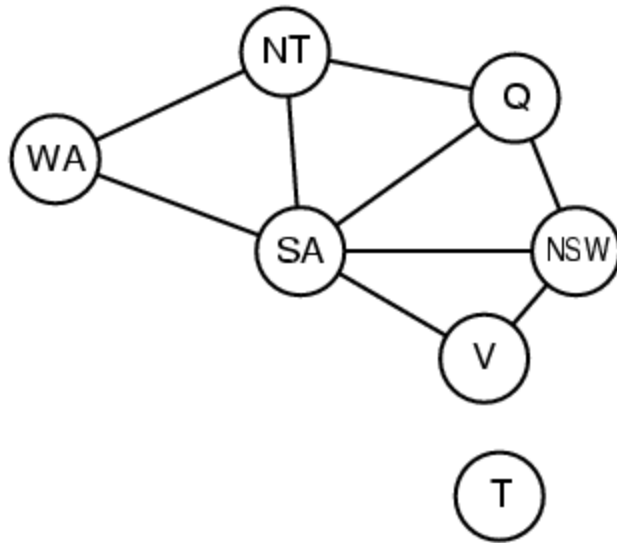
Matching itself can be expensive:

Database indexing allows $O(1)$ retrieval of known facts

– e.g., query $Missile(x)$ retrieves $Missile(M_1)$

Forward chaining is widely used in **deductive databases**

Hard matching example


$$\begin{aligned} & \text{Diff}(wa, nt) \wedge \text{Diff}(wa, sa) \wedge \text{Diff}(nt, q) \wedge \\ & \text{Diff}(nt, sa) \wedge \text{Diff}(q, nsw) \wedge \text{Diff}(q, sa) \wedge \\ & \text{Diff}(nsw, v) \wedge \text{Diff}(nsw, sa) \wedge \text{Diff}(v, sa) \Rightarrow \\ & \text{Colorable()} \end{aligned}$$
$$\begin{array}{ll} \text{Diff}(\text{Red}, \text{Blue}) & \text{Diff}(\text{Red}, \text{Green}) \\ \text{Diff}(\text{Green}, \text{Red}) & \text{Diff}(\text{Green}, \text{Blue}) \\ \text{Diff}(\text{Blue}, \text{Red}) & \text{Diff}(\text{Blue}, \text{Green}) \end{array}$$

- *Colorable()* is inferred iff the CSP has a solution
- CSPs include 3SAT as a special case, hence matching is NP-hard

Backward chaining algorithm

function FOL-BC-ASK(KB , goals, θ) returns a set of substitutions

inputs: KB , a knowledge base

$goals$, a list of conjuncts forming a query

θ , the current substitution, initially the empty substitution $\{ \}$

local variables: ans , a set of substitutions, initially empty

if $goals$ is empty **then return** $\{ \theta \}$

$q' \leftarrow \text{SUBST}(\theta, \text{FIRST}(goals))$

for each r in KB where $\text{STANDARDIZE-APART}(r) = (p_1 \wedge \dots \wedge p_n \Rightarrow q)$
and $\theta' \leftarrow \text{UNIFY}(q, q')$ succeeds

$ans \leftarrow \text{FOL-BC-ASK}(KB, [p_1, \dots, p_n | \text{REST}(goals)], \text{COMPOSE}(\theta, \theta')) \cup ans$

return ans

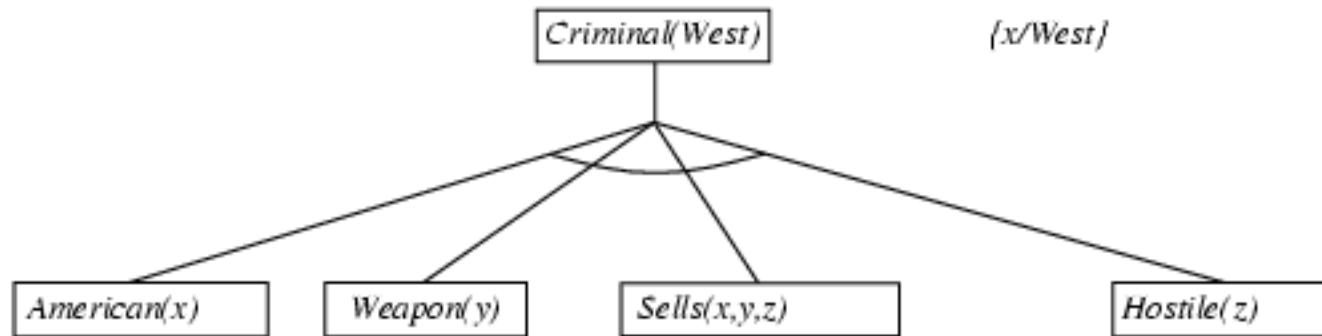
$$\text{SUBST}(\text{COMPOSE}(\theta_1, \theta_2), p) = \text{SUBST}(\theta_2, \text{SUBST}(\theta_1, p))$$

Backward chaining example

Criminal(West)

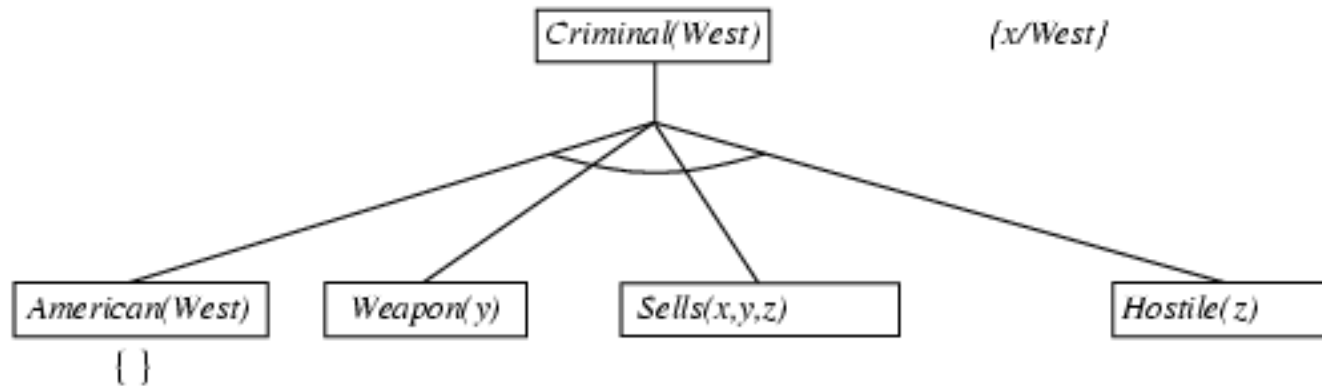
R1: $\text{American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}(x,y,z) \wedge \text{Hostile}(z) \Rightarrow \text{Criminal}(x)$

Backward chaining example



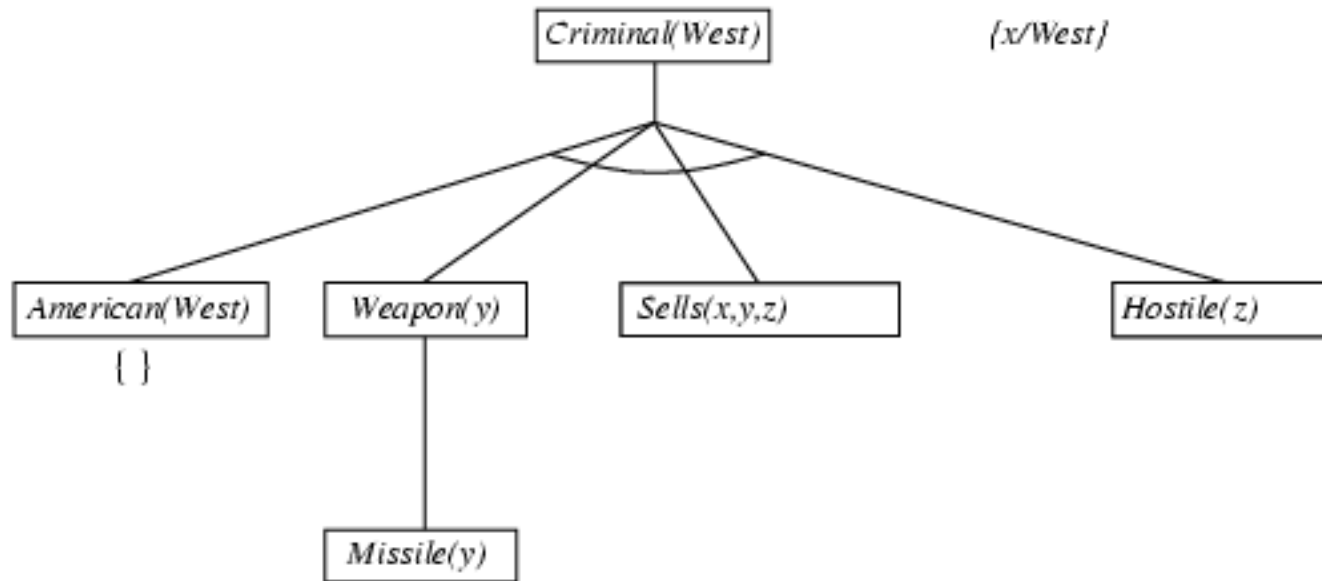
R1: $American(x) \wedge Weapon(y) \wedge Sells(x,y,z) \wedge Hostile(z) \Rightarrow Criminal(x)$

Backward chaining example



R1: $\text{American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}(x,y,z) \wedge \text{Hostile}(z) \Rightarrow \text{Criminal}(x)$

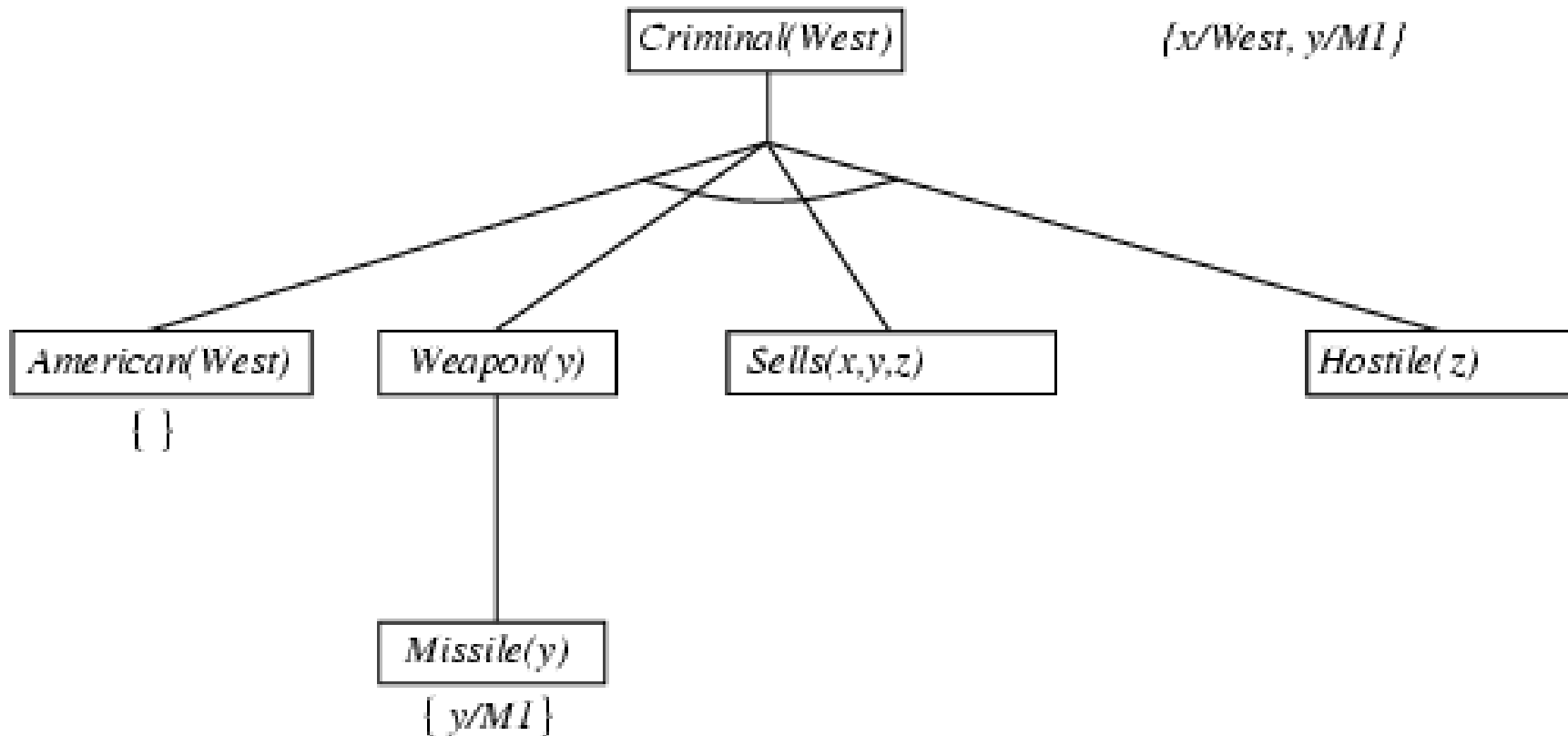
Backward chaining example



R4: $\text{Missile}(x) \Rightarrow \text{Weapon}(x)$

R1: $\text{American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}(x,y,z) \wedge \text{Hostile}(z) \Rightarrow \text{Criminal}(x)$

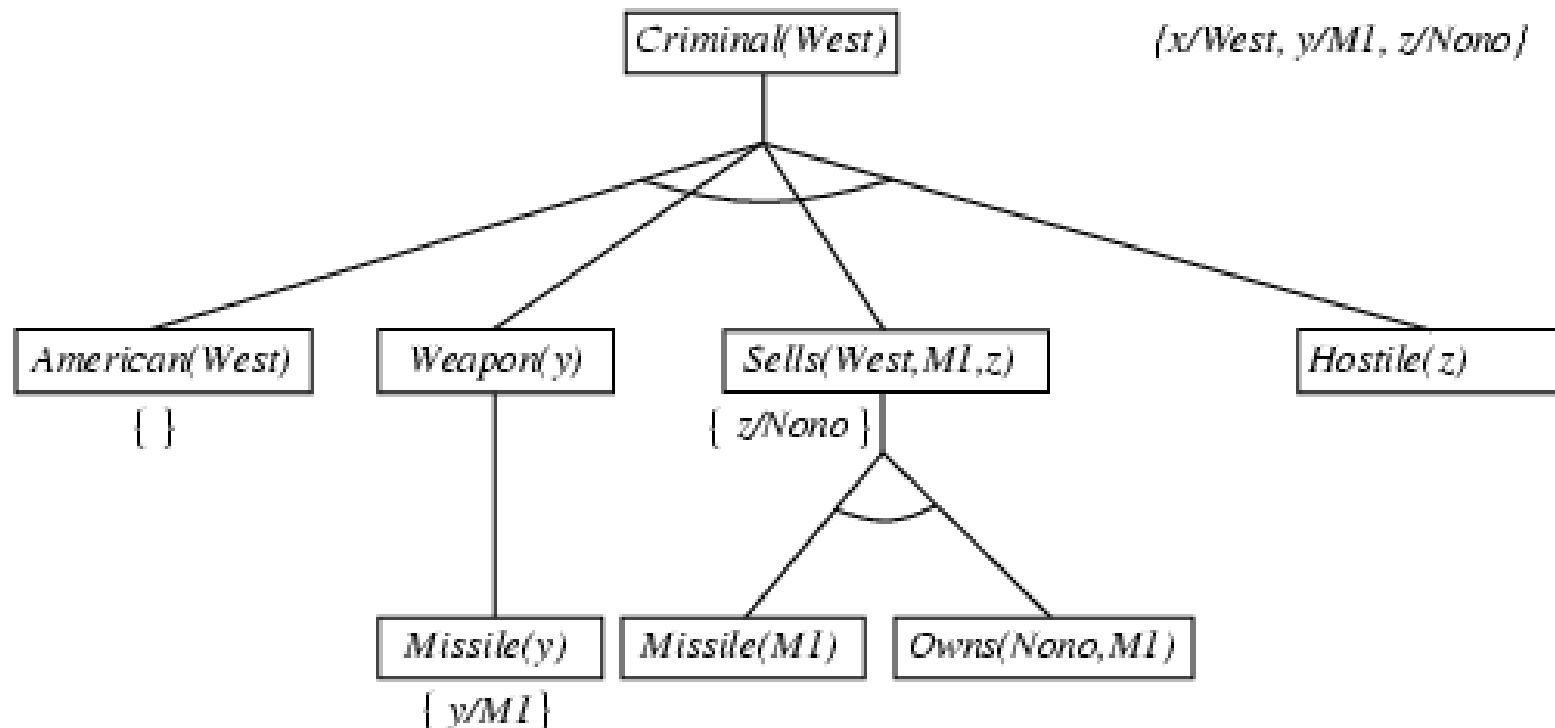
Backward chaining example



R4: $\text{Missile}(x) \Rightarrow \text{Weapon}(x)$

R1: $\text{American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}(x,y,z) \wedge \text{Hostile}(z) \Rightarrow \text{Criminal}(x)$

Backward chaining example

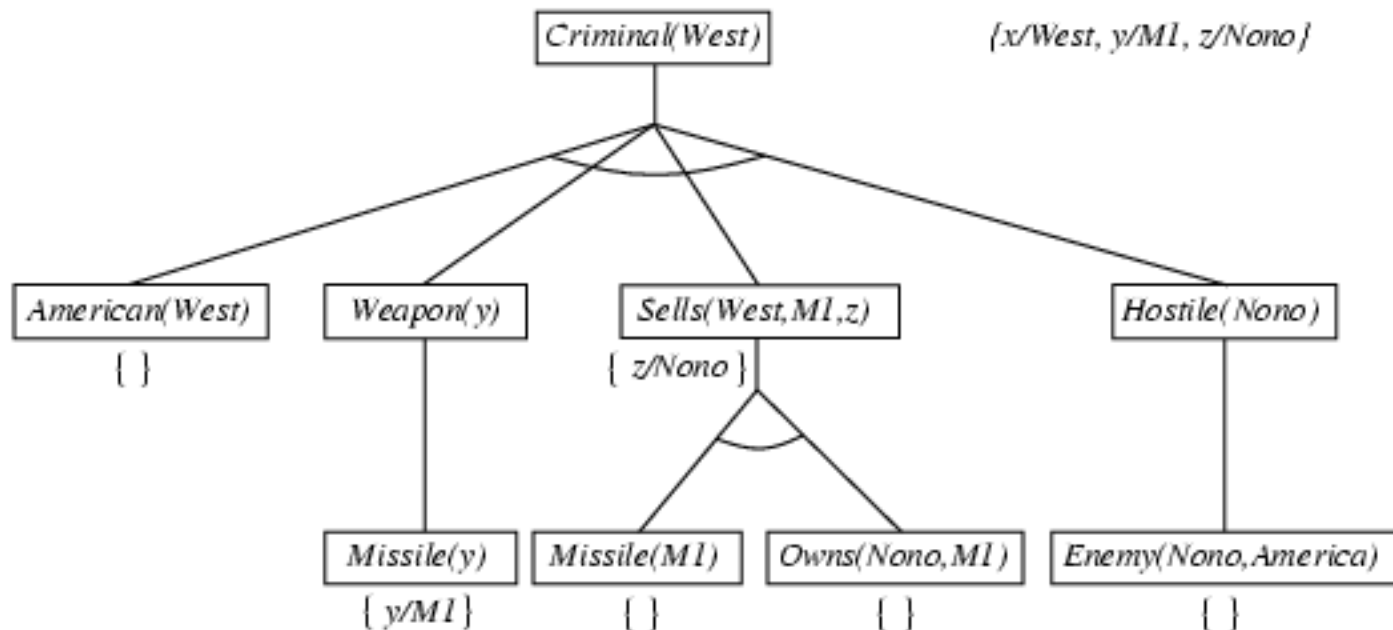


R3: $Missile(x) \wedge Owns(Nono, x) \Rightarrow Sells(West, x, Nono)$

R4: $Missile(x) \Rightarrow Weapon(x)$

R1: $American(x) \wedge Weapon(y) \wedge Sells(x, y, z) \wedge Hostile(z) \Rightarrow Criminal(x)$

Backward chaining example



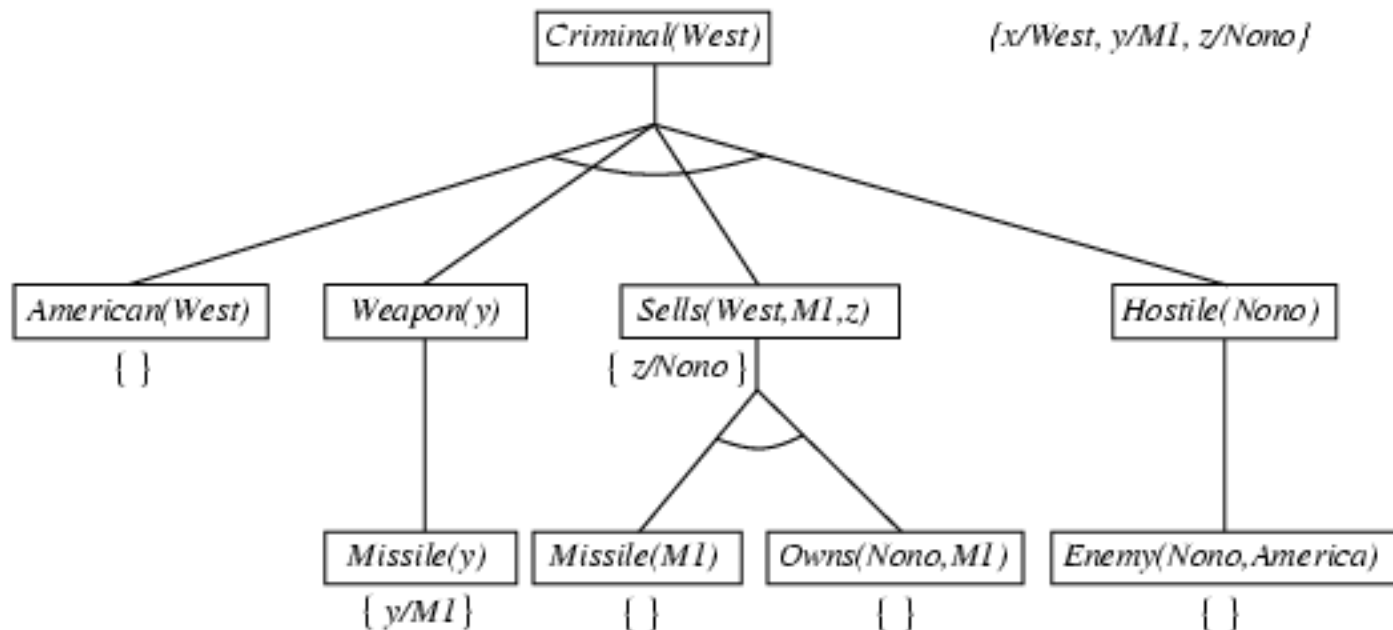
R5: $\text{Enemy}(x, \text{America}) \Rightarrow \text{Hostile}(x)$

R3: $\text{Missile}(x) \wedge \text{Owns}(\text{Nono}, x) \Rightarrow \text{Sells}(\text{West}, x, \text{Nono})$

R4: $\text{Missile}(x) \Rightarrow \text{Weapon}(x)$

R1: $\text{American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}(x, y, z) \wedge \text{Hostile}(z) \Rightarrow \text{Criminal}(x)$

Backward chaining example



R5: $\text{Enemy}(x, \text{America}) \Rightarrow \text{Hostile}(x)$

R3: $\text{Missile}(x) \wedge \text{Owns}(\text{Nono}, x) \Rightarrow \text{Sells}(\text{West}, x, \text{Nono})$

R4: $\text{Missile}(x) \Rightarrow \text{Weapon}(x)$

R1: $\text{American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}(x, y, z) \wedge \text{Hostile}(z) \Rightarrow \text{Criminal}(x)$

Properties of backward chaining

- **Depth-first recursive proof search:** space is linear in size of proof
- Incomplete due to infinite loops
⇒ fix by checking current goal against every goal on stack
- Inefficient due to repeated subgoals (both success and failure)
⇒ fix using caching of previous results (extra space)
- Widely used for **logic programming**

Logic programming: Prolog

- Algorithm = Logic + Control
- Basis: backward chaining with Horn clauses + bells & whistles
Widely used in Europe, Japan (basis of 5th Generation project)
Compilation techniques $\Rightarrow$ 60 million LIPS
- Program = set of clauses = `head :- literal1, ... literaln.`

`criminal(X) :- american(X), weapon(Y), sells(X,Y,Z), hostile(Z).`
- Depth-first, left-to-right backward chaining
- Built-in predicates for arithmetic etc., e.g., `X is Y*Z+3`
- Built-in predicates that have side effects (e.g., input and output predicates, assert/retract predicates)
- Closed-world assumption ("negation as failure")
 - e.g., given `alive(X) :- not dead(X).`
 - `alive(joe)` succeeds if `dead(joe)` fails

Prolog

- Appending two lists to produce a third:

```
append([ ], Y, Y) .
```

```
append([X|L], Y, [X|Z]) :- append(L, Y, Z) .
```

- query: `append(A, B, [1, 2]) ?`

- answers: `A = [ ]        B = [1, 2]`

```
A = [1]        B = [2]
```

```
A = [1, 2]    B = [ ]
```

Resolution: brief summary

- Full first-order version:

$$\frac{\ell_1 \vee \cdots \vee \ell_k, \quad m_1 \vee \cdots \vee m_n}{(\ell_1 \vee \cdots \vee \ell_{i-1} \vee \ell_{i+1} \vee \cdots \vee \ell_k \vee m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \vee \cdots \vee m_n)\theta}$$

where $\text{Unify}(\ell_i, \neg m_j) = \theta$.

- The two clauses are assumed to be standardized apart so that they share no variables.
- For example,

$$\frac{\neg \text{Rich}(x) \vee \text{Unhappy}(x) \quad \text{Rich}(\text{Ken})}{\text{Unhappy}(\text{Ken})}$$

with $\theta = \{x/\text{Ken}\}$

- Apply resolution steps to $\text{CNF}(\text{KB} \wedge \neg \alpha)$; complete for FOL

Conversion to CNF

- Everyone who loves all animals is loved by someone:

$$\forall x [\forall y \text{ Animal}(y) \Rightarrow \text{Loves}(x,y)] \Rightarrow [\exists y \text{ Loves}(y,x)]$$

- 1. Eliminate biconditionals and implications

$$\forall x [\neg \forall y \neg \text{Animal}(y) \vee \text{Loves}(x,y)] \vee [\exists y \text{ Loves}(y,x)]$$

- 2. Move $\neg$ inwards: $\neg \forall x p \equiv \exists x \neg p$, $\neg \exists x p \equiv \forall x \neg p$

$$\forall x [\exists y \neg (\neg \text{Animal}(y) \vee \text{Loves}(x,y))] \vee [\exists y \text{ Loves}(y,x)]$$

$$\forall x [\exists y \neg \neg \text{Animal}(y) \wedge \neg \text{Loves}(x,y)] \vee [\exists y \text{ Loves}(y,x)]$$

$$\forall x [\exists y \text{ Animal}(y) \wedge \neg \text{Loves}(x,y)] \vee [\exists y \text{ Loves}(y,x)]$$

Conversion to CNF contd.

3. Standardize variables: each quantifier should use a different one

$$\forall x [\exists y \textit{Animal}(y) \wedge \neg \textit{Loves}(x,y)] \vee [\exists z \textit{Loves}(z,x)]$$

4. Skolemize: a more general form of existential instantiation.
Each existential variable is replaced by a **Skolem function** of the enclosing universally quantified variables:

$$\forall x [\textit{Animal}(F(x)) \wedge \neg \textit{Loves}(x,F(x))] \vee \textit{Loves}(G(x),x)$$

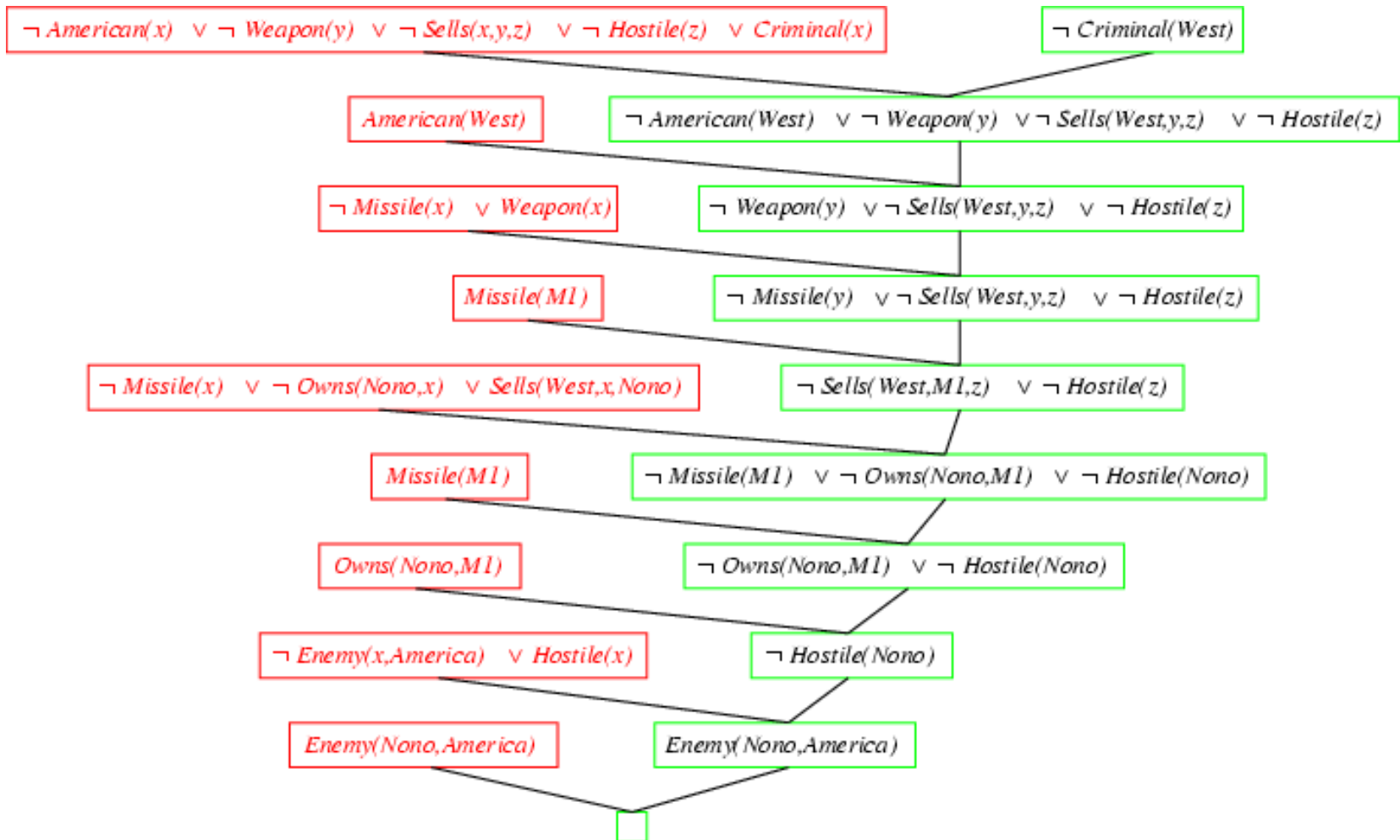
5. Drop universal quantifiers:

$$[\textit{Animal}(F(x)) \wedge \neg \textit{Loves}(x,F(x))] \vee \textit{Loves}(G(x),x)$$

6. Distribute $\vee$ over $\wedge$:

$$[\textit{Animal}(F(x)) \vee \textit{Loves}(G(x),x)] \wedge [\neg \textit{Loves}(x,F(x)) \vee \textit{Loves}(G(x),x)]$$

Resolution proof: definite clauses





University of Colorado
Colorado Springs



University of Colorado

Boulder | Colorado Springs | Denver | Anschutz Medical Campus